

Python String Interview Questions

30 most asked questions — every solution from brute force to optimal

How to Use This File

Each question follows the same structure:

- **Problem** — clear statement with example input and output.
 - **Brute Force** — the naive approach; correct but slow. Understand this first.
 - **Better** — an improved approach with reduced complexity.
 - **Optimal** — the best known solution; this is what you present in an interview.
 - **Complexity** — time and space for the optimal solution.
 - **Interview Tip** — what the interviewer actually wants to hear.
-

Q1 — Reverse a String

Problem: Given a string, return it reversed.

Input: "python"

Output: "nohtyp"

```
# Brute Force -- build result character by character
def reverse_brute(s: str) -> str:
    result = ""
    for i in range(len(s) - 1, -1, -1):
        result += s[i]          # creates a new string object on every iteration
    return result

print(reverse_brute("python"))  # 'nohtyp'
```

```
# Better -- use a list to avoid repeated string creation, then join
def reverse_better(s: str) -> str:
    chars = list(s)
    left, right = 0, len(chars) - 1
    while left < right:
        chars[left], chars[right] = chars[right], chars[left]
```

```

        left += 1
        right -= 1
    return "".join(chars)

print(reverse_better("python")) # 'nohtyp'

```

```

# Optimal -- Python slice; O(n) time, O(n) space, single expression
def reverse_string(s: str) -> str:
    return s[::-1]

print(reverse_string("python")) # 'nohtyp'

```

Complexity: Time $O(n)$ — Space $O(n)$

Interview Tip: Mention all three approaches. Say "in Python the idiomatic choice is slicing, but in a language without slicing I would use the two-pointer swap on a character array."

Q2 — Check if a String is a Palindrome

Problem: Return `True` if the string reads the same forward and backward.

```

Input:  "racecar"    → True
Input:  "hello"      → False

```

```

# Brute Force -- reverse and compare
def is_palindrome_brute(s: str) -> bool:
    return s == s[::-1]

print(is_palindrome_brute("racecar")) # True
print(is_palindrome_brute("hello"))   # False

```

```

# Better -- two-pointer, stops early on first mismatch
def is_palindrome_two_pointer(s: str) -> bool:
    left, right = 0, len(s) - 1
    while left < right:
        if s[left] != s[right]:
            return False
        left += 1
        right -= 1

```

```
        return True

print(is_palindrome_two_pointer("racecar"))  # True
```

```
# Optimal -- two-pointer with case and punctuation ignored (real interview variant)
def is_palindrome(s: str) -> bool:
    cleaned = "".join(ch.lower() for ch in s if ch.isalnum())
    left, right = 0, len(cleaned) - 1
    while left < right:
        if cleaned[left] != cleaned[right]:
            return False
        left += 1
        right -= 1
    return True

print(is_palindrome("A man, a plan, a canal: Panama"))  # True
print(is_palindrome("race a car"))                      # False
```

Complexity: Time $O(n)$ — Space $O(n)$ for cleaned string

Interview Tip: Always ask "should I ignore spaces, punctuation, and case?" Interviewers at FAANG almost always mean the cleaned version.

Q3 — Check if Two Strings are Anagrams

Problem: Return `True` if both strings contain exactly the same characters with the same frequencies.

```
Input:  "listen", "silent"  → True
Input:  "hello",  "world"   → False
```

```
# Brute Force -- sort both strings and compare
def are_anagrams_brute(s1: str, s2: str) -> bool:
    return sorted(s1) == sorted(s2)    #  $O(n \log n)$ 

print(are_anagrams_brute("listen", "silent"))  # True
```

```
# Better -- frequency count using a dict
def are_anagrams_dict(s1: str, s2: str) -> bool:
    if len(s1) != len(s2):
        return False
    freq: dict[str, int] = {}
    for ch in s1:
        freq[ch] = freq.get(ch, 0) + 1
    for ch in s2:
        freq[ch] = freq.get(ch, 0) - 1
        if freq[ch] < 0:
            return False
    return True

print(are_anagrams_dict("listen", "silent")) # True
```

```
# Optimal -- Counter comparison; clean and O(n)
from collections import Counter

def are_anagrams(s1: str, s2: str) -> bool:
    return Counter(s1) == Counter(s2)

print(are_anagrams("listen", "silent")) # True
print(are_anagrams("hello", "world")) # False
```

Complexity: Time $O(n)$ — Space $O(1)$ (bounded by alphabet size, at most 26 keys)

Interview Tip: The `Counter` approach is readable and fast. Mention that if only lowercase letters are guaranteed, a fixed-size array of 26 integers is even more space-efficient.

Q4 — Count Vowels and Consonants

Problem: Count the number of vowels and consonants in a string.

```
Input: "education"
Output: vowels = 5, consonants = 4
```

```
# Brute Force -- loop and check membership in a list
def count_vowels_brute(s: str) -> tuple[int, int]:
    vowels = 0
```

```

consonants = 0
for ch in s.lower():
    if ch in ['a', 'e', 'i', 'o', 'u']:
        vowels += 1
    elif ch.isalpha():
        consonants += 1
return vowels, consonants

```

```

# Optimal -- set lookup is O(1) per character; generator expression
def count_vowels(s: str) -> tuple[int, int]:
    vowel_set = set("aeiou")
    cleaned = s.lower()
    vowels = sum(1 for ch in cleaned if ch in vowel_set)
    consonants = sum(1 for ch in cleaned if ch.isalpha() and ch not in vowel_set)
    return vowels, consonants

v, c = count_vowels("education")
print(f"vowels -- {v}, consonants -- {c}") # vowels -- 5, consonants -- 4

```

Complexity: Time $O(n)$ — Space $O(1)$

Interview Tip: Replace list membership check with a `set` — this is a very common optimisation point interviewers look for.

Q5 — Find the First Non-Repeating Character

Problem: Return the first character in the string that appears exactly once. Return `''` if none exists.

```

Input:  "aabbcdde"  → 'c'
Input:  "aabb"      → ''

```

```

# Brute Force -- two nested loops, O(n²)
def first_unique_brute(s: str) -> str:
    for i in range(len(s)):
        found = True
        for j in range(len(s)):
            if i != j and s[i] == s[j]:
                found = False
                break
    return s[i] if found else ''

```

```
        if found:
            return s[i]
    return ''
```

Better -- use a frequency dict, then scan again

```
def first_unique_dict(s: str) -> str:
    freq: dict[str, int] = {}
    for ch in s:
        freq[ch] = freq.get(ch, 0) + 1
    for ch in s:
        if freq[ch] == 1:
            return ch
    return ''
```

Optimal -- Counter + single scan to preserve order

```
from collections import Counter

def first_unique(s: str) -> str:
    freq = Counter(s)
    for ch in s:
        if freq[ch] == 1:
            return ch
    return ''

print(first_unique("aabbccde")) # 'c'
print(first_unique("aabb"))     # ''
```

Complexity: Time $O(n)$ — Space $O(1)$ (at most 26 keys for lowercase letters)

Interview Tip: Walk through the two-pass approach clearly: first build frequency, then find the first with count 1.

Q6 — Remove Duplicate Characters

Problem: Remove all duplicate characters from a string, keeping only the first occurrence.

```
Input:  "programming"
Output: "progamin"
```

```
# Brute Force -- check if character already seen using a list
def remove_duplicates_brute(s: str) -> str:
    seen = []
    result = ""
    for ch in s:
        if ch not in seen:    # O(n) per lookup -- slow
            seen.append(ch)
            result += ch
    return result
```

```
# Optimal -- use a set for O(1) lookup; preserve insertion order
def remove_duplicates(s: str) -> str:
    seen: set[str] = set()
    result: list[str] = []
    for ch in s:
        if ch not in seen:
            seen.add(ch)
            result.append(ch)
    return "".join(result)

print(remove_duplicates("programming"))  # 'progamin'
```

Complexity: Time $O(n)$ — Space $O(n)$

Interview Tip: Python `dict.fromkeys()` also preserves insertion order and gives a one-liner:

`"".join(dict.fromkeys(s))`. Know both.

```
# One-liner using dict.fromkeys (Python 3.7+ ordered dict guaranteed)
def remove_duplicates_oneliner(s: str) -> str:
    return "".join(dict.fromkeys(s))

print(remove_duplicates_oneliner("programming"))  # 'progamin'
```

Q7 — Count Occurrences of Each Character

Problem: Return a dictionary of each character and how many times it appears.

Input: "banana"

```
Output: {'b': 1, 'a': 3, 'n': 2}
```

```
# Brute Force -- manual dict
def char_count_brute(s: str) -> dict[str, int]:
    freq: dict[str, int] = {}
    for ch in s:
        if ch in freq:
            freq[ch] += 1
        else:
            freq[ch] = 1
    return freq
```

```
# Better -- dict.get()
def char_count_get(s: str) -> dict[str, int]:
    freq: dict[str, int] = {}
    for ch in s:
        freq[ch] = freq.get(ch, 0) + 1
    return freq
```

```
# Optimal -- Counter
from collections import Counter

def char_count(s: str) -> dict[str, int]:
    return dict(Counter(s))

print(char_count("banana"))  # {'b': 1, 'a': 3, 'n': 2}
```

Complexity: Time $O(n)$ — Space $O(k)$ where k is the number of unique characters

Interview Tip: `Counter` also gives you `.most_common(n)` for free — useful for follow-up questions.

Q8 — Reverse Words in a Sentence

Problem: Reverse the order of words. Multiple spaces between words should be reduced to one.

```
Input: "  hello world  "
Output: "world hello"
```



```
# Brute Force -- split on spaces manually
def reverse_words_brute(s: str) -> str:
    words = s.split(' ')
    words = [w for w in words if w != '']
    result = ''
    for i in range(len(words) - 1, -1, -1):
        result += words[i]
        if i != 0:
            result += ' '
    return result
```

```
# Optimal -- split (handles multiple spaces), reverse, join
def reverse_words(s: str) -> str:
    return " ".join(s.split()[::-1])

print(reverse_words("  hello world  "))      # 'world hello'
print(reverse_words("learning Python is fun")) # 'fun is Python learning'
```

Complexity: Time $O(n)$ — Space $O(n)$

Interview Tip: `s.split()` without an argument strips leading/trailing spaces and handles multiple spaces automatically — mention this explicitly.

Q9 — Check if a String is a Rotation of Another

Problem: Return `True` if `s2` is a rotation of `s1`.

```
Input: s1 = "abcde", s2 = "cdeab" → True
Input: s1 = "abcde", s2 = "abced" → False
```

```
# Brute Force -- try every rotation
def is_rotation_brute(s1: str, s2: str) -> bool:
    if len(s1) != len(s2):
        return False
    n = len(s1)
    for i in range(n):
        rotated = s1[i:] + s1[:i]
```

```
    if rotated == s2:
        return True
    return False
```

Optimal -- concatenate s1 with itself; if s2 is a rotation it must appear as a substring

```
def is_rotation(s1: str, s2: str) -> bool:
    if len(s1) != len(s2):
        return False
    return s2 in (s1 + s1)

print(is_rotation("abcde", "cdeab"))  # True
print(is_rotation("abcde", "abced"))  # False
```

Complexity: Time $O(n)$ — Space $O(n)$

Interview Tip: This is a classic trick. The key insight: if `s2` is a rotation of `s1`, then `s2` will always be a substring of `s1 + s1`. State the insight before writing code.

Q10 — Longest Common Prefix

Problem: Find the longest common prefix among a list of strings.

```
Input:  ["flower", "flow", "flight"]
Output: "fl"
```

```
Input:  ["dog", "racecar", "car"]
Output: ""
```

Brute Force -- compare character by character across all strings

```
def lcp_brute(strs: list[str]) -> str:
    if not strs:
        return ""
    prefix = ""
    for i in range(len(strs[0])):
        ch = strs[0][i]
        for s in strs[1:]:
            if i >= len(s) or s[i] != ch:
```

```

        return prefix
    prefix += ch
    return prefix

```

```

# Better -- use zip to iterate columns
def lcp_zip(strs: list[str]) -> str:
    if not strs:
        return ""
    prefix = []
    for chars in zip(*strs):
        if len(set(chars)) == 1:
            prefix.append(chars[0])
        else:
            break
    return "".join(prefix)

```

```

# Optimal -- sort and compare only first and last string
def longest_common_prefix(strs: list[str]) -> str:
    if not strs:
        return ""
    strs.sort()
    first, last = strs[0], strs[-1]
    i = 0
    while i < len(first) and i < len(last) and first[i] == last[i]:
        i += 1
    return first[:i]

print(longest_common_prefix(["flower", "flow", "flight"])) # 'fl'
print(longest_common_prefix(["dog", "racecar", "car"]))    # ''

```

Complexity: Time $O(n \log n + m)$ where m is length of shortest string — Space $O(1)$

Interview Tip: Explain why sorting works: the lexicographically smallest and largest strings in a sorted list will have the least in common, so comparing just those two is sufficient.

Q11 — Count Occurrences of a Substring

Problem: Count how many times a substring appears in a string (non-overlapping).

Input: s = "mississippi", sub = "issi"

Output: 1

Input: s = "aaaa", sub = "aa"

Output: 2

Brute Force -- slide through and compare manually

```
def count_substr_brute(s: str, sub: str) -> int:
    count = 0
    n, m = len(s), len(sub)
    for i in range(n - m + 1):
        if s[i:i + m] == sub:
            count += 1
    return count
```

Optimal -- built-in count() handles non-overlapping correctly

```
def count_substr(s: str, sub: str) -> int:
    return s.count(sub)

print(count_substr("mississippi", "issi")) # 1
print(count_substr("aaaa", "aa")) # 2
```

Bonus -- overlapping count using find() in a loop

```
def count_overlapping(s: str, sub: str) -> int:
    count, start = 0, 0
    while (pos := s.find(sub, start)) != -1:
        count += 1
        start = pos + 1
    return count

print(count_overlapping("aaaa", "aa")) # 3
```

Complexity: Time $O(n * m)$ for brute, $O(n)$ for built-in — Space $O(1)$

Interview Tip: Know the difference between overlapping and non-overlapping counting — interviewers often ask for both.

Q12 — Check if a String Contains Only Digits

Problem: Return `True` if every character in the string is a digit.

Input: "123456" → True
Input: "123a56" → False

```
# Brute Force -- loop and check each character
def all_digits_brute(s: str) -> bool:
    for ch in s:
        if not ch.isdigit():
            return False
    return True
```

```
# Better -- all() with generator
def all_digits_gen(s: str) -> bool:
    return all(ch.isdigit() for ch in s)
```

```
# Optimal -- built-in isdigit()
def all_digits(s: str) -> bool:
    return s.isdigit() and len(s) > 0

print(all_digits("123456")) # True
print(all_digits("123a56")) # False
print(all_digits("")) # False
```

Complexity: Time $O(n)$ — Space $O(1)$

Interview Tip: Note the edge case of an empty string — `"".isdigit()` returns `False` in Python, but always state your assumption about empty input.

Q13 — Find All Positions of a Substring

Problem: Return a list of all starting indices where a substring appears.

Input: s = "ababab", sub = "ab"
Output: [0, 2, 4]

```
# Brute Force -- scan every position
def find_all_brute(s: str, sub: str) -> list[int]:
    positions = []
    n, m      = len(s), len(sub)
    for i in range(n - m + 1):
        if s[i:i + m] == sub:
            positions.append(i)
    return positions
```

```
# Optimal -- use find() with a moving start pointer
def find_all(s: str, sub: str) -> list[int]:
    positions = []
    start     = 0
    while (pos := s.find(sub, start)) != -1:
        positions.append(pos)
        start = pos + 1    # +1 to allow overlapping; use +len(sub) for non-overlapping
    return positions

print(find_all("ababab", "ab"))    # [0, 2, 4]
print(find_all("aaaaaa", "aa"))   # [0, 1, 2, 3, 4]
```

Complexity: Time $O(n * m)$ average — Space $O(k)$ where k is the number of matches

Interview Tip: The walrus operator `:=` is a clean way to assign and test in one step here — interviewers notice this.

Q14 — Longest Substring Without Repeating Characters

Problem: Find the length of the longest substring with no repeated characters.

```
Input:  "abcabcb"  → 3  ("abc")
Input:  "bbbbbb"   → 1  ("b")
Input:  "pwwkew"   → 3  ("wke")
```

```
# Brute Force -- check all substrings, O(n^3)
def longest_unique_brute(s: str) -> int:
    n = len(s)
    max_len = 0
```

```

for i in range(n):
    for j in range(i + 1, n + 1):
        if len(set(s[i:j])) == j - i:    # all characters unique
            max_len = max(max_len, j - i)
return max_len

```

```

# Better -- sliding window with a set, O(n)
def longest_unique_set(s: str) -> int:
    seen = set()
    left = 0
    max_len = 0
    for right in range(len(s)):
        while s[right] in seen:
            seen.discard(s[left])
            left += 1
        seen.add(s[right])
        max_len = max(max_len, right - left + 1)
    return max_len

```

```

# Optimal -- sliding window with a dict storing last seen index; O(n), fewer iterations
def longest_unique(s: str) -> int:
    last_seen: dict[str, int] = {}
    max_len = 0
    left = 0
    for right, ch in enumerate(s):
        if ch in last_seen and last_seen[ch] >= left:
            left = last_seen[ch] + 1    # jump left past the duplicate
        last_seen[ch] = right
        max_len = max(max_len, right - left + 1)
    return max_len

print(longest_unique("abcabcbb"))    # 3
print(longest_unique("bbbbbb"))     # 1
print(longest_unique("pwwkew"))     # 3

```

Complexity: Time $O(n)$ — Space $O(\min(n, \text{alphabet_size}))$

Interview Tip: This is one of the most common FAANG string questions. Explain the three approaches in order and why the dict version is faster than the set version (no inner while loop needed).

Q15 — Check if Two Strings are Isomorphic

Problem: Two strings are isomorphic if characters in one can be replaced to get the other, with a consistent one-to-one mapping.

```
Input: "egg", "add" → True   (e→a, g→d)
Input: "foo", "bar" → False  (o maps to both a and r)
Input: "paper", "title" → True
```

```
# Brute Force -- try all character mappings (impractical)
# Not shown -- exponential complexity.
```

```
# Better -- single mapping dict (fails the reverse check)
def is_isomorphic_partial(s: str, t: str) -> bool:
    mapping: dict[str, str] = {}
    for cs, ct in zip(s, t):
        if cs in mapping:
            if mapping[cs] != ct:
                return False
        else:
            mapping[cs] = ct
    return True
# Bug: is_isomorphic_partial("ba", "aa") → True (wrong)
```

```
# Optimal -- maintain both forward and reverse mappings
def is_isomorphic(s: str, t: str) -> bool:
    s_to_t: dict[str, str] = {}
    t_to_s: dict[str, str] = {}
    for cs, ct in zip(s, t):
        if (cs in s_to_t and s_to_t[cs] != ct) or \
            (ct in t_to_s and t_to_s[ct] != cs):
            return False
        s_to_t[cs] = ct
        t_to_s[ct] = cs
    return True
```



```
print(is_isomorphic("egg", "add"))    # True
print(is_isomorphic("foo", "bar"))    # False
print(is_isomorphic("paper", "title")) # True
```

Complexity: Time $O(n)$ — Space $O(1)$ (bounded by alphabet size)

Interview Tip: The classic mistake is using only one-direction mapping. Always check both directions. State this explicitly before coding.

Q16 — String Compression (Run-Length Encoding)

Problem: Compress consecutive repeated characters. If compression does not reduce length, return the original.

Input: "aaabbbccdddee"

Output: "a3b3c2d4e2"

Input: "abc"

Output: "abc" (no benefit from compressing)

```
# Brute Force -- sort and count (wrong -- changes order)
# Not applicable here; sorted approach destroys order.
```

```
# Better -- count consecutive runs
def compress_better(s: str) -> str:
    if not s:
        return s
    result = ""
    count = 1
    for i in range(1, len(s)):
        if s[i] == s[i - 1]:
            count += 1
        else:
            result += s[i - 1] + str(count)
            count = 1
    result += s[-1] + str(count)
    return result if len(result) < len(s) else s
```

Optimal -- use a list to avoid $O(n^2)$ string concatenation

```
def compress(s: str) -> str:
    if not s:
        return s
    parts = []
    count = 1
    for i in range(1, len(s)):
        if s[i] == s[i - 1]:
            count += 1
        else:
            parts.append(s[i - 1] + str(count))
            count = 1
    parts.append(s[-1] + str(count))
    compressed = "".join(parts)
    return compressed if len(compressed) < len(s) else s

print(compress("aaabbbccdddee"))  # 'a3b3c2d4e2'
print(compress("abc"))             # 'abc'
```

Complexity: Time $O(n)$ — Space $O(n)$

Interview Tip: Always mention the condition "return original if compressed is longer or equal."

Interviewers test this edge case.

Q17 — Sort Characters by Frequency

Problem: Return the string with characters sorted by decreasing frequency.

Input: "tree"
Output: "eert" or "eetr" (both valid)

Input: "cccaaa"
Output: "cccaaa" or "aaaccc"

```
# Brute Force -- manual sort
def sort_by_freq_brute(s: str) -> str:
    freq: dict[str, int] = {}
    for ch in s:
        freq[ch] = freq.get(ch, 0) + 1
```

```
chars = list(freq.keys())
chars.sort(key=lambda c: -freq[c])
return "".join(c * freq[c] for c in chars)
```

```
# Optimal -- Counter.most_common()
from collections import Counter

def sort_by_freq(s: str) -> str:
    return "".join(ch * count for ch, count in Counter(s).most_common())

print(sort_by_freq("tree"))      # 'eert'
print(sort_by_freq("cccaaa"))    # 'cccaaa'
print(sort_by_freq("Aabb"))      # 'bbAa'
```

Complexity: Time $O(n \log n)$ — Space $O(n)$

Interview Tip: `Counter.most_common()` returns pairs in descending order. Knowing this avoids writing a custom sort.

Q18 — Check for Pangram

Problem: A pangram is a sentence that contains every letter of the alphabet at least once. Return `True` if the string is a pangram.

```
Input: "The quick brown fox jumps over the lazy dog" → True
Input: "Hello world" → False
```

```
# Brute Force -- check each letter of the alphabet separately
def is_pangram_brute(s: str) -> bool:
    s = s.lower()
    for ch in 'abcdefghijklmnopqrstuvwxyz':
        if ch not in s:
            return False
    return True
```

```
# Optimal -- set comparison
def is_pangram(s: str) -> bool:
    return set('abcdefghijklmnopqrstuvwxyz').issubset(set(s.lower()))
```

```
# Even shorter
def is_pangram_v2(s: str) -> bool:
    return len(set(s.lower()) & set('abcdefghijklmnopqrstuvwxyz')) == 26

print(is_pangram("The quick brown fox jumps over the lazy dog")) # True
print(is_pangram("Hello world")) # False
```

Complexity: Time $O(n)$ — Space $O(1)$ (set size bounded by 26)

Interview Tip: Use `issubset()` — it reads naturally as English and is $O(1)$ for sets of fixed size.

Q19 — Decode/Expand a Run-Length Encoded String

Problem: Decode a run-length encoded string.

```
Input:  "a4b3c2"
Output: "aaaabbbcc"
```

```
# Brute Force -- scan character by character with a flag
def decode_brute(s: str) -> str:
    result = ""
    i = 0
    while i < len(s):
        ch = s[i]
        if i + 1 < len(s) and s[i + 1].isdigit():
            result += ch * int(s[i + 1])
            i += 2
        else:
            result += ch
            i += 1
    return result
```

```
# Optimal -- group letter with following digits using enumerate
def decode(s: str) -> str:
    result = []
    prev_ch = ""
    for ch in s:
        if ch.isalpha():
            result.append(ch)
            prev_ch = ch
```

```

        else:
            result.append(prev_ch * (int(ch) - 1))
    return "".join(result)

print(decode("a4b3c2"))  # 'aaaabbbcc'

```

```

# Alternative using regex -- handles multi-digit numbers
import re

def decode_regex(s: str) -> str:
    return "".join(ch * int(num) for ch, num in re.findall(r'([a-zA-Z])(\d+)',
s))

print(decode_regex("a4b3c12"))  # 'aaaabbbccccccccccccc'

```

Complexity: Time $O(n + \text{output_length})$ — Space $O(\text{output_length})$

Interview Tip: The regex approach handles multi-digit numbers cleanly. Always ask "can counts be more than 9?" before writing single-digit-only code.

Q20 — Find the Longest Palindromic Substring

Problem: Return the longest substring that is a palindrome.

```

Input:  "babad"  → "bab"   or "aba"
Input:  "cbbd"   → "bb"

```

```

# Brute Force -- check all substrings, O(n³)
def longest_palindrome_brute(s: str) -> str:
    n      = len(s)
    result = s[0] if s else ""
    for i in range(n):
        for j in range(i + 1, n + 1):
            sub = s[i:j]
            if sub == sub[::-1] and len(sub) > len(result):
                result = sub
    return result

```

```

# Optimal -- expand around centre, O(n^2) time O(1) space
def longest_palindrome(s: str) -> str:
    if not s:
        return ""

    def expand(left: int, right: int) -> str:
        while left >= 0 and right < len(s) and s[left] == s[right]:
            left -= 1
            right += 1
        return s[left + 1:right]

    result = ""
    for i in range(len(s)):
        odd = expand(i, i) # odd-length palindromes
        even = expand(i, i + 1) # even-length palindromes
        if len(odd) > len(result): result = odd
        if len(even) > len(result): result = even
    return result

print(longest_palindrome("babad")) # 'bab'
print(longest_palindrome("cbbd")) # 'bb'
print(longest_palindrome("racecar")) # 'racecar'

```

Complexity: Time $O(n^2)$ — Space $O(1)$

Interview Tip: Mention that Manacher's algorithm solves this in $O(n)$ but is complex to implement under interview pressure. The expand-around-centre approach is the expected answer in most interviews.

Q21 — Count Words in a String

Problem: Count the number of words in a string. Words are separated by one or more spaces.

```

Input: "  hello world  python  "
Output: 3

```

```

# Brute Force -- manually track word boundaries
def count_words_brute(s: str) -> int:
    count = 0
    in_word = False
    for ch in s:

```

```

        if ch != ' ' and not in_word:
            count += 1
            in_word = True
        elif ch == ' ':
            in_word = False
    return count

```

```

# Optimal -- split handles all whitespace edge cases
def count_words(s: str) -> int:
    return len(s.split())

print(count_words("  hello world  python  ")) # 3
print(count_words("")) # 0
print(count_words("  ")) # 0

```

Complexity: Time $O(n)$ — Space $O(n)$ for the split list

Interview Tip: Note that `s.split()` without arguments is always better than `s.split(' ')` for this task because it handles multiple consecutive spaces and leading/trailing spaces correctly.

Q22 — Reverse Internal Content of Each Word

Problem: Reverse the characters inside each word, keeping word order intact.

```

Input:  "Hello World Python"
Output: "olleH dlrow nohtyP"

```

```

# Brute Force -- loop with index
def reverse_each_word_brute(s: str) -> str:
    words = s.split()
    result = []
    for w in words:
        rev = ""
        for ch in w:
            rev = ch + rev
        result.append(rev)
    return " ".join(result)

```

```
# Optimal -- list comprehension with slice
def reverse_each_word(s: str) -> str:
    return " ".join(w[::-1] for w in s.split())

print(reverse_each_word("Hello World Python")) # 'olleH dlroW nohtyP'
```

Complexity: Time $O(n)$ — Space $O(n)$

Q23 — Check if String has Balanced Parentheses

Problem: Return `True` if every opening bracket has a matching closing bracket in the correct order.

```
Input:  "(()())"  → True
Input:  "("       → False
Input:  ")("      → False
```

```
# Brute Force -- repeatedly remove matched pairs until nothing changes
def is_balanced_brute(s: str) -> bool:
    prev = None
    while prev != s:
        prev = s
        s = s.replace("()", "")
    return s == ""
```

```
# Optimal -- stack-based; O(n) time and space
def is_balanced(s: str) -> bool:
    count = 0 # works as a simple stack counter for one bracket type
    for ch in s:
        if ch == '(':
            count += 1
        elif ch == ')':
            count -= 1
        if count < 0: # closing before opening
            return False
    return count == 0
```



```
print(is_balanced("(()())"))  # True
print(is_balanced("("))      # False
print(is_balanced(""))       # False
```

General version for multiple bracket types using an actual stack

```
def is_balanced_general(s: str) -> bool:
    stack = []
    mapping = {')': '(', ']': '[', '}': '{'}
    for ch in s:
        if ch in mapping:
            top = stack.pop() if stack else '#'
            if mapping[ch] != top:
                return False
        elif ch in '([{':
            stack.append(ch)
    return not stack

print(is_balanced_general("{[()] }"))  # True
print(is_balanced_general("{[()] }"))  # False
```

Complexity: Time $O(n)$ — Space $O(n)$

Interview Tip: Always present the general multi-bracket version. It shows broader thinking.

Q24 — Implement strStr() — Find Substring Index

Problem: Return the index of the first occurrence of `needle` in `haystack`, or -1 if not found. (LeetCode 28)

```
Input: haystack = "sadbutsad", needle = "sad" → 0
Input: haystack = "leetcode", needle = "leeto" → -1
```

*# Brute Force -- slide needle over haystack, $O(n * m)$*

```
def str_str_brute(haystack: str, needle: str) -> int:
    n, m = len(haystack), len(needle)
    for i in range(n - m + 1):
        if haystack[i:i + m] == needle:
            return i
    return -1
```

```
# Built-in -- Python's find() uses a highly optimised internal algorithm
def str_str_builtin(haystack: str, needle: str) -> int:
    return haystack.find(needle)
```

```
# Optimal for interviews -- KMP (Knuth-Morris-Pratt), O(n + m)
def str_str_kmp(haystack: str, needle: str) -> int:
    if not needle:
        return 0
    n, m = len(haystack), len(needle)

    # Build failure table
    fail = [0] * m
    j = 0
    for i in range(1, m):
        while j > 0 and needle[i] != needle[j]:
            j = fail[j - 1]
        if needle[i] == needle[j]:
            j += 1
        fail[i] = j

    # Search
    j = 0
    for i in range(n):
        while j > 0 and haystack[i] != needle[j]:
            j = fail[j - 1]
        if haystack[i] == needle[j]:
            j += 1
        if j == m:
            return i - m + 1
    return -1

print(str_str_kmp("sadbutsad", "sad"))    # 0
print(str_str_kmp("leetcode", "leeto")) # -1
```

Complexity: Time $O(n + m)$ — Space $O(m)$

Interview Tip: For most interviews, the brute force is accepted. Mention KMP exists and explain its idea — you do not need to code it perfectly from memory, but knowing it exists is impressive.

Q25 — Convert Integer to Roman Numeral

Problem: Convert a positive integer to its Roman numeral representation.

Input: 3749
Output: "MMMDCCXLIX"

```
# Brute Force -- manually handle each digit position  
# Impractical -- requires 40 separate if/elif conditions.
```

```
# Optimal -- greedy with a value-symbol table  
def int_to_roman(num: int) -> str:  
    values = [1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, 4, 1]  
    symbols = ["M", "CM", "D", "CD", "C", "XC", "L", "XL", "X", "IX", "V", "IV", "I"]  
    result = []  
    for value, symbol in zip(values, symbols):  
        while num >= value:  
            result.append(symbol)  
            num -= value  
    return "".join(result)  
  
print(int_to_roman(3749))  ## 'MMMDCCXLIX'  
print(int_to_roman(58))   ## 'LVIII'  
print(int_to_roman(1994)) ## 'MCMXCIV'
```

Complexity: Time $O(1)$ — Space $O(1)$ (bounded by the number of symbols)

Interview Tip: The greedy approach works because Roman numerals are constructed by always taking the largest possible symbol. State this before coding.

Q26 — Roman Numeral to Integer

Problem: Convert a Roman numeral string to an integer.

Input: "MCMXCIV" → 1994
Input: "LVIII" → 58

```
# Brute Force -- process left to right, add or subtract
def roman_to_int_brute(s: str) -> int:
    roman = {'I': 1, 'V': 5, 'X': 10, 'L': 50,
             'C': 100, 'D': 500, 'M': 1000}
    result = 0
    for i in range(len(s)):
        if i + 1 < len(s) and roman[s[i]] < roman[s[i + 1]]:
            result -= roman[s[i]]
        else:
            result += roman[s[i]]
    return result
```

```
# Optimal -- process right to left; add if current >= previous, else subtract
def roman_to_int(s: str) -> int:
    roman = {'I': 1, 'V': 5, 'X': 10, 'L': 50,
             'C': 100, 'D': 500, 'M': 1000}
    result = roman[s[-1]]
    for i in range(len(s) - 2, -1, -1):
        if roman[s[i]] < roman[s[i + 1]]:
            result -= roman[s[i]]
        else:
            result += roman[s[i]]
    return result

print(roman_to_int("MCMXCIV")) # 1994
print(roman_to_int("LVIII")) # 58
```

Complexity: Time $O(n)$ — Space $O(1)$

Interview Tip: The key rule: if a smaller value appears before a larger value, it is subtracted. State this rule out loud before writing code.

Q27 — Minimum Window Substring

Problem: Find the smallest window in `s` that contains all characters of `t`.

```
Input: s = "ADOBECODEBANC", t = "ABC"
Output: "BANC"
```

```

# Brute Force -- check all substrings,  $O(n^2 * m)$ 
def min_window_brute(s: str, t: str) -> str:
    from collections import Counter
    need = Counter(t)
    result = ""
    n = len(s)
    for i in range(n):
        for j in range(i + len(t), n + 1):
            window = s[i:j]
            if all(Counter(window)[ch] >= need[ch] for ch in need):
                if not result or len(window) < len(result):
                    result = window
    return result

```

```

# Optimal -- sliding window with two frequency counters,  $O(n + m)$ 
from collections import Counter

def min_window(s: str, t: str) -> str:
    if not s or not t:
        return ""
    need = Counter(t)
    window = {}
    have = 0
    required = len(need)
    best = ""
    left = 0

    for right, ch in enumerate(s):
        window[ch] = window.get(ch, 0) + 1
        if ch in need and window[ch] == need[ch]:
            have += 1

        while have == required:
            # Record best window
            candidate = s[left:right + 1]
            if not best or len(candidate) < len(best):
                best = candidate
            # Shrink from left
            left_ch = s[left]
            window[left_ch] -= 1

```

```

        if left_ch in need and window[left_ch] < need[left_ch]:
            have -= 1
        left += 1

    return best

print(min_window("ADOBECODEBANC", "ABC"))  # 'BANC'
print(min_window("a", "a"))                # 'a'
print(min_window("a", "aa"))                # ''

```

Complexity: Time $O(n + m)$ — Space $O(n + m)$

Interview Tip: This is a hard-level question that appears at Google and Meta. Explain the sliding window idea first — expand right until valid, then shrink left until invalid, record minimum at each valid state.

Q28 — Group Anagrams Together

Problem: Given a list of strings, group the anagrams together.

```

Input:  ["eat", "tea", "tan", "ate", "nat", "bat"]
Output: [["bat"], ["nat", "tan"], ["ate", "eat", "tea"]]

```

```

# Brute Force -- compare every pair,  $O(n^2 * m \log m)$ 
def group_anagrams_brute(words: list[str]) -> list[list[str]]:
    visited = [False] * len(words)
    groups = []
    for i in range(len(words)):
        if not visited[i]:
            group = [words[i]]
            for j in range(i + 1, len(words)):
                if not visited[j] and sorted(words[i]) == sorted(words[j]):
                    group.append(words[j])
                    visited[j] = True
            groups.append(group)
    return groups

```

```

# Optimal -- sort each word as key;  $O(n * m \log m)$ 
from collections import defaultdict

def group_anagrams(words: list[str]) -> list[list[str]]:

```

```

groups: dict[str, list[str]] = defaultdict(list)
for word in words:
    key = "".join(sorted(word))
    groups[key].append(word)
return list(groups.values())

print(group_anagrams(["eat", "tea", "tan", "ate", "nat", "bat"]))
# [['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]

```

```

# Alternative key -- tuple of character counts; avoids sorting
def group_anagrams_count(words: list[str]) -> list[list[str]]:
    groups: dict[tuple, list[str]] = defaultdict(list)
    for word in words:
        count = [0] * 26
        for ch in word:
            count[ord(ch) - ord('a')] += 1
        groups[tuple(count)].append(word)
    return list(groups.values())

```

Complexity: Sorted key — $O(n * m \log m)$ time. Count key — $O(n * m)$ time. Both $O(n * m)$ space.

Interview Tip: Present both key strategies. The count-based key is $O(n * m)$ vs $O(n * m \log m)$ — a noticeable improvement for long words.

Q29 — Validate an IP Address

Problem: Return `"IPv4"`, `"IPv6"`, or `"Neither"` for the given string.

```

Input:  "192.168.1.1"           → "IPv4"
Input:  "2001:0db8:85a3:0000:0000:8a2e:0370:7334" → "IPv6"
Input:  "256.100.0.1"          → "Neither"

```

```

# Brute Force -- many nested if/else checks

```

```

# Optimal -- split on delimiter and validate each part
def validate_ip(s: str) -> str:
    def is_valid_ipv4(s: str) -> bool:
        parts = s.split('.')
        if len(parts) != 4:

```

```

        return False
    for part in parts:
        if not part or not part.isdigit():
            return False
        if part != str(int(part)):    # rejects leading zeros like "01"
            return False
        if not (0 <= int(part) <= 255):
            return False
    return True

def is_valid_ipv6(s: str) -> bool:
    parts = s.split(':')
    if len(parts) != 8:
        return False
    valid_hex = set('0123456789abcdefABCDEF')
    for part in parts:
        if not (1 <= len(part) <= 4):
            return False
        if not all(ch in valid_hex for ch in part):
            return False
    return True

if is_valid_ipv4(s):
    return "IPv4"
elif is_valid_ipv6(s):
    return "IPv6"
return "Neither"

print(validate_ip("192.168.1.1"))           # 'IPv4'
print(validate_ip("2001:0db8:85a3:0000:0000:8a2e:0370:7334")) # 'IPv6'
print(validate_ip("256.100.0.1"))           # 'Neither'
print(validate_ip("192.168.01.1"))          # 'Neither' (leading zero)

```

Complexity: Time $O(1)$ — Space $O(1)$ (inputs are bounded in length)

Interview Tip: Leading zero detection (`part != str(int(part))`) is the most commonly missed edge case. Mention it proactively.

Q30 — Longest Repeating Character Replacement

Problem: You can replace at most `k` characters in a string to make a substring where all characters are the same. Return the length of the longest such substring.

Input: `s = "AABABBA", k = 1 → 4`

Input: `s = "ABAB", k = 2 → 4`

```
# Brute Force -- check all substrings, O(n³)
def char_replacement_brute(s: str, k: int) -> int:
    n = len(s)
    result = 0
    for i in range(n):
        for j in range(i, n):
            substr = s[i:j + 1]
            max_freq = max(substr.count(ch) for ch in set(substr))
            if (j - i + 1) - max_freq <= k:
                result = max(result, j - i + 1)
    return result
```

```
# Optimal -- sliding window with frequency tracking, O(n)
from collections import defaultdict

def char_replacement(s: str, k: int) -> int:
    freq = defaultdict(int)
    max_freq = 0
    left = 0
    result = 0

    for right in range(len(s)):
        freq[s[right]] += 1
        max_freq = max(max_freq, freq[s[right]])

        # window size - most frequent char count > k means we need to shrink
        window_size = right - left + 1
        if window_size - max_freq > k:
            freq[s[left]] -= 1
            left += 1

    result = max(result, right - left + 1)
```

```
    return result

print(char_replacement("AABABBA", 1))  ## 4
print(char_replacement("ABAB", 2))    ## 4
print(char_replacement("AAAA", 2))    ## 4
```

Complexity: Time $O(n)$ — Space $O(1)$ (at most 26 keys)

Interview Tip: The key insight is: a window is valid if `window_size - max_frequency_in_window <= k`. We never need to shrink `max_freq` below its highest value because we are looking for the maximum window. State this insight clearly before coding.

Complexity Summary Table

#	PROBLEM	BRUTE FORCE	OPTIMAL	KEY TECHNIQUE
1	Reverse string	$O(n^2)$	$O(n)$	Slice <code>[::-1]</code>
2	Palindrome check	$O(n)$	$O(n)$	Two-pointer
3	Anagram check	$O(n \log n)$	$O(n)$	Counter
4	Count vowels	$O(n)$	$O(n)$	Set membership
5	First non-repeating	$O(n^2)$	$O(n)$	Counter + scan
6	Remove duplicates	$O(n^2)$	$O(n)$	Set for seen
7	Char frequency	$O(n)$	$O(n)$	Counter
8	Reverse words	$O(n)$	$O(n)$	split + slice
9	String rotation	$O(n^2)$	$O(n)$	Concatenate and <code>in</code>
10	Longest common prefix	$O(n * m)$	$O(n \log n + m)$	Sort and compare ends
11	Count substring	$O(n * m)$	$O(n)$	Built-in <code>count()</code>
12	All digits check	$O(n)$	$O(n)$	<code>isdigit()</code>
13	All positions of substr	$O(n * m)$	$O(n * m)$	<code>find()</code> with walrus
14	Longest unique substring	$O(n^3)$	$O(n)$	Sliding window + dict
15	Isomorphic strings	$O(n^2)$	$O(n)$	Two-way mapping
16	String compression	$O(n^2)$	$O(n)$	Scan + list join
17	Sort by frequency	$O(n \log n)$	$O(n \log n)$	<code>Counter.most_common()</code>
18	Pangram check	$O(n)$	$O(n)$	Set <code>issubset()</code>
19	Decode RLE string	$O(n)$	$O(n)$	Scan + <code>chr()</code>
20	Longest palindromic substr	$O(n^3)$	$O(n^2)$	Expand around centre
21	Count words	$O(n)$	$O(n)$	<code>split()</code>
22	Reverse each word	$O(n)$	$O(n)$	List comp + slice
23	Balanced parentheses	$O(n^2)$	$O(n)$	Stack / counter

#	PROBLEM	BRUTE FORCE	OPTIMAL	KEY TECHNIQUE
24	Find substring (strStr)	$O(n * m)$	$O(n + m)$	KMP algorithm
25	Integer to Roman	$O(l)$	$O(l)$	Greedy table
26	Roman to Integer	$O(n)$	$O(n)$	Subtract if smaller before larger
27	Minimum window substring	$O(n^2 * m)$	$O(n + m)$	Sliding window + Counter
28	Group anagrams	$O(n^2 * m)$	$O(n * m \log m)$	Sort as key
29	Validate IP address	$O(l)$	$O(l)$	Split and validate
30	Longest char replacement	$O(n^3)$	$O(n)$	Sliding window + max_freq

Interview Strategy

Follow this order every time a string problem is asked:

1. **Clarify** — ask about case sensitivity, spaces, special characters, empty input, encoding (ASCII or Unicode).
2. **State brute force** — say it out loud even if you will not code it. It shows you understand the problem.
3. **Identify the pattern** — most string problems use one of: two-pointer, sliding window, frequency counter, or sorting as a key.
4. **Code the optimal** — write clean code with meaningful variable names.
5. **Test with examples** — trace through the given examples and at least one edge case (empty string, single character, all same characters).
6. **State complexity** — always state time and space complexity at the end.

PATTERN	WHEN TO USE
Two-pointer	Palindrome, reverse, comparison from both ends
Sliding window	Longest/shortest substring with a condition
Frequency counter	Anagram, first unique, sorting by frequency
Hashing (dict)	Isomorphic strings, group anagrams, index tracking
Sort as key	Grouping, prefix matching
Stack	Balanced brackets, nested structure validation